Drift-Aware MLOps Pipeline for EV Charge-Session Energy and Dwell-Time Prediction

Technical report

Programme	upGrad MLOps, August 2025 batch — Bring Your Own Capstone
Author	Deva Prakash J
Repository (the submission)	https://github.com/deva-prakash-j/ev-charge-mlops
Dataset	ACN-Data (Caltech / PowerFlex): 66,745 real charging sessions, 3 sites, Apr 2018 – Sep 2021
Report date	20 October 2026

All figures and tables in this report are aggregate metrics computed on the licensed extract. No licensed session records are reproduced here or committed to the repository.

Contents

1. Executive Summary 2

2. Introduction & Business Context 3

3. Data Architecture & Methodology 3

4. Implementation & Results 5

5. Strategic Deductions & Business Impact 9

6. Limitations & Future Work 10

Appendix A. Reproduction 11

Appendix B. References 11

Appendix C. Detailed V2 architecture 12

1. Executive Summary

The problem. Workplace EV charging sites predict, at plug-in, how much energy a session will draw and how long the car will stay. Those two numbers drive the site load forecast that keeps the site under its contracted demand limit, the "ready-by" time shown to drivers, and the smart-charging schedule. A model trained once decays silently when the fleet and occupancy change: the API stays up and every uptime dashboard stays green while the forecasts drift.

The approach. I built an end-to-end, drift-aware MLOps pipeline on 66,745 real ACN-Data sessions. It includes a 16-check data contract with quarantine, a shared feature builder for training and serving, chronological evaluation against mean and Ridge floors, and MLflow tracking with an alias-based promotion gate. The model is served by FastAPI in Docker, and GitHub Actions CI proves a degraded candidate is blocked. A monthly Prefect flow scores Evidently PSI/KS/JS drift into Prometheus and Grafana. The detector was validated against a real, dated regime change (the April 2020 workplace shutdown), with thresholds declared before scoring.

Result	Measured value	What it means
Energy prediction quality	MAE 4.40 kWh vs 7.89 naive (−44%)	Less headroom to hold back per plug-in
Dwell (ready-by) quality	MAE 1.66 h vs 3.07 naive (−46%)	Departure estimates ~1.4 h closer
Silent failure exposed	Error +43% (6.27 kWh) after April 2020	Invisible to uptime and latency monitoring
Drift detector	April 2020 PSI 0.231 > 0.20; 0/8 false alarms	Alerts on real shift, quiet in stable months
Release safety	Degraded candidate (13.5 kWh) blocked in CI	Bad models cannot reach the scheduler
Rollback	Alias + previous-image drill passed	Known-good version restorable without retraining
Recovery target	Best retrain 5.71 kWh vs ≤ 4.84 target	**Not met** — reported as a null result

Recommendation. Adopt the monitoring, promotion gate and rollback controls now: they turn a silent failure into a dated, auditable alert and stop regressions at release time. Treat predictions for the smallest site (0019) as advisory only. Do not rely on automatic retraining to restore accuracy after a regime change until site-aware or recency-weighted training and the deferred occupancy feature have been evaluated; the measured recovery gap is 29.8% above the pre-shift error.

2. Introduction & Business Context

2.1 Why the prediction matters

Commercial charging sites are billed for energy and for **demand**: the highest short-interval power draw in the billing month. A site that under-forecasts its load breaches its contracted limit and pays a demand charge set by that single peak, or has to curtail charging abruptly. A site that over-forecasts strands capacity it has already paid for and turns drivers away. Between those two failure modes sits the smart-charging scheduler, which divides limited power across occupied bays using the predicted energy and departure time of each car. Drivers judge the service on one number, the ready-by estimate; when it is wrong, they stop trusting the app.

2.2 The operational stake: silent degradation

The model's inputs are not stationary. In March–April 2020 workplace charging collapsed: monthly sessions fell 86% at site 0001 and 97% at site 0002 against the reference-period monthly median. The sessions that remained looked different, and mean delivered energy at site 0002 rose from 7.33 to 11.79 kWh. A frozen model kept answering every request with HTTP 200 and flat latency while its error rose 43%. Conventional service monitoring cannot see this failure, because it lives in the data, not the infrastructure.

2.3 Prior approaches and their pitfalls

Common industry practice	Why it fails	Response in this project
Retrain on a calendar (e.g. monthly)	Retrains whether or not anything changed; cannot say whether the new model is better	Retraining is triggered by a validated drift alert and must pass a promotion gate
Monitor uptime, latency and error rate	Blind to distribution shift; the April 2020 failure is fully invisible	Per-feature PSI/KS/JS, target and prediction drift exported to Prometheus with alert rules
Treat offline accuracy as a guarantee	Random splits leak across regime changes and flatter the score	Strictly chronological train/validation/post-shift evaluation; post-shift score reported
Re-implement transforms in the service	Train/serve skew silently corrupts predictions	One feature module used by both paths; 1,000-session parity audit in the test suite

2.4 Consumers and ownership

Consumer	Decision supported	What they need from the model
Charging operations / site energy manager	Demand-limit headroom, site load forecast	Unbiased energy forecast; early warning when it degrades
Charging app / driver-experience team	Ready-by estimate shown to the driver	Dwell-time prediction with known error
Smart-charging scheduler	Power allocation across occupied bays	Per-session energy and departure inputs at plug-in
Data governance / audit	Provenance and release approval	Data fingerprints, PII audit, promotion decision log

The **MLOps engineer owns** deployment, promotion, rollback, monitoring, alert response and retraining operations. **Charging operations accepts or rejects** retrained candidates before promotion. For this academic deployment, Deva Prakash J is the acting owner and first responder. A production deployment would need a staffed on-call rota and a named operations approver (see `docs/operations_governance.md`).

2.5 Success criteria declared in advance

Before scoring, I registered the hypotheses and thresholds in `docs/drift_hypotheses.md` and did not revise them afterwards. H1: delivered-energy PSI > 0.2 within one monthly batch of the 2020 shutdown, with a ≥ 70% volume fall at sites 0001 and 0002. H2: a ≥ 20% rise in the 90th percentile of requested energy (fleet-mix proxy). H3: a shift in the declared-vs-delivered gap (PSI > 0.2 or ≥ 2 kWh median change). Recovery: retraining must return post-shift MAE to within 10% of the pre-shift validation MAE.

3. Data Architecture & Methodology

3.1 Source, licence and provenance

ACN-Data (Caltech with PowerFlex) publishes real sessions from Adaptive Charging Networks. I used three sites: 0001 (Caltech campus), 0002 (NASA JPL) and 0019 (an office site). That gives 66,745 sessions from April 2018 to September 2021, retrieved through the authenticated API. Registration binds users to educational and research

use, and the publisher requests a citation (Lee, Li and Low, e-Energy '19). The terms do not address redistribution.

Decision: the repository is data-free. The extract is not committed, because republishing it would bypass the publisher's registration and usage tracking. Instead the repository tracks a manifest with per-site SHA-256 hashes, byte counts and session counts, the retrieval code, and benchmark files that pin the training-frame fingerprint and exact scores. Evaluators reproduce the licensed results with their own free token. A 2,579-row synthetic fixture, generated from aggregate statistics only, runs every stage without credentials. A pre-submission history audit found identifier values from one real session used as unit-test inputs; they were replaced and the finding is disclosed in `DATA_LICENCE.md`.

3.2 Privacy and data quality

A structure-aware PII audit (committed as `artifacts/pii_audit.json`) classifies every field. It found no direct PII; `userID` is a pseudonymous publisher identifier. The audit recognises Mongo IDs, dashed station IDs and float representations, so it does not raise false phone or card matches. A Great Expectations contract with 16 checks gates training, and offending rows are quarantined with a reason rather than silently dropped.

Defect found in the real feed	Scale	Handling
`siteID` emitted as both int 2 and string "0002"	2,081 rows	Coerced; blocking expectation
`clusterID` mixed int/string	Crashed Parquet write	Coerced; blocking expectation
Sessions of 245 h and 214 h	2 rows	Quarantined with reason (66,743 clean rows)
Timestamp order, station namespace, declared/delivered plausibility	16 checks in total	Site-specific domains; documented tolerances

3.3 Leakage controls and features

The model predicts at plug-in time, so only information available at that moment may enter the feature matrix. `userInputs` is a list of driver revisions, and only the earliest entry is used. `disconnectTime`, `doneChargingTime` and the targets are declared post-hoc in `configs/data.yaml` and rejected by the feature builder and the API. Features fall into three groups:

- **Declared intent:** `kWhRequested`, `milesRequested`, `WhPerMile`, `minutesAvailable`, plus the derived declared duration and declared kWh per minute. A `has_user_inputs` flag keeps the 27% of sessions with no request explicit, and missing values stay NaN.
- **Calendar:** arrival hour (raw and sine/cosine), day of week, month and weekend flag, all in the site's local time.
- **Location:** site, cluster and station codes from a vocabulary fitted on training data only; unseen values map to an explicit unknown code.

Rolling site occupancy was proposed but is **deferred**. It needs a point-in-time as-of join that has not yet passed a parity and leakage audit, and shipping it unaudited would risk exactly the skew this project guards against.

3.4 Evaluation design

Split	Window (local time)	Rows	Purpose
Train	2018-04 to 2019-12	48,957	Fit models and feature vocabulary
Validation	2020-01 to 2020-02	4,815	Model selection and promotion gate
(excluded)	2020-03	—	Transition month, not used for scoring
Post-shift	2020-04 to 2020-06	942	Same fitted model scored after the regime change

The split is temporal, never random, because a random split would leak post-shift behaviour into training. Every candidate must beat two floors, the training mean and a Ridge regression, so added complexity has to earn its place. The candidate model is scikit-learn's HistGradientBoostingRegressor, which handles missing declared inputs natively. Errors are reported overall and by site.

3.5 Architecture decisions

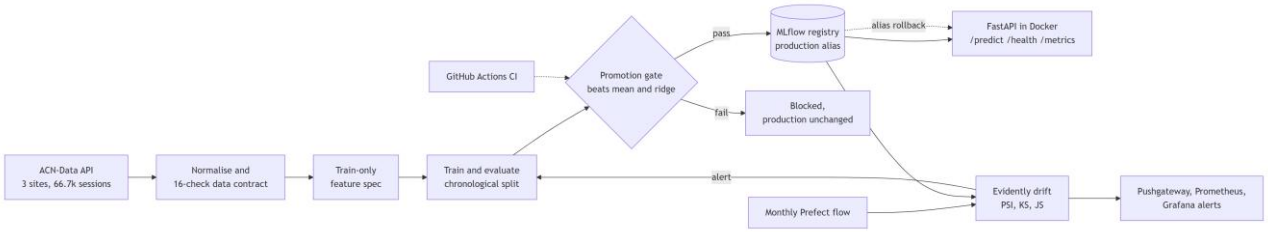


Figure 1. End-to-end control loop (simplified). The detailed V2 component diagram is in Appendix C.

Decision	Alternative rejected	Justification
HistGradientBoosting	LightGBM (proposal)	LightGBM crashes the process on Windows when pyarrow loads first (OpenMP runtime conflict); same algorithm family, native NaN support
Great Expectations contract	Pandera, ad-hoc asserts	Serialisable per-expectation results give auditable evidence for quarantine decisions
MLflow aliases `staging/production`	MLflow stages, W&B	Stages are deprecated since MLflow 2.9; local-only constraint rules out hosted W&B
FastAPI in Docker Compose	Kubernetes	One-host academic scope; Compose gives the registry, API and monitoring stack reproducibly
Prefect 3	Airflow, cron	Python-native flows with retries and schedules; lighter than Airflow for one monthly job
Evidently + Pushgateway + Prometheus + Grafana	Custom scripts	Standard drift statistics; Pushgateway suits short-lived batch jobs; alert rules are declarative
Deterministic multiplicative shifts	SDV synthetic data	Known injected magnitude makes the sensitivity curve exact and reproducible
Data-free repository + manifests	Commit the extract	Respects publisher registration while keeping provenance verifiable by hash

4. Implementation & Results

4.1 Pipeline and experiment tracking

One command, `python -m evcharge.pipeline --source raw|sample`, runs normalise → contract → train → evaluate → register. Each MLflow run logs parameters, the git commit, the SHA-256 of the post-quarantine training frame, split row counts, overall and per-site metrics, permutation importances, and the fitted feature specification as an artifact. A model is never stored without the encoding it was fitted with. Energy (`kWhDelivered`) and dwell (`connectedHours`) are trained as separate targets on the same plug-in-time features.

4.2 Model quality

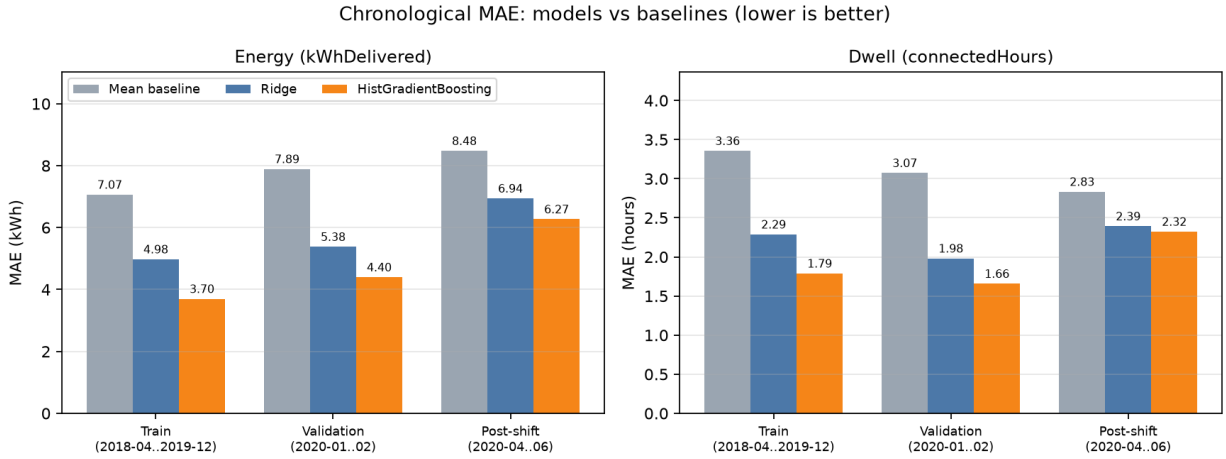


Figure 2. Mean absolute error by model and chronological split (licensed extract).

Energy model (kWh)	Train MAE	Validation MAE	Post-shift MAE	Validation R^2	Post-shift R^2
Mean floor	7.067	7.894	8.479	−0.010	−0.169
Ridge	4.979	5.382	6.944	0.501	0.235
HistGradientBoosting	**3.701**	**4.399**	**6.272**	**0.579**	**0.355**

Table: energy model, licensed extract (benchmarks/licensed_baseline.json). Validation RMSE 7.15 kWh; median absolute error 2.38 kWh.

The selected model beats the mean floor by **44%** and Ridge by **18%** on validation. Scored on post-shift data, the same model's error rises **43%** (4.40 → 6.27 kWh) and R^2 falls from 0.579 to 0.355. This is the failure the monitoring stack exists to expose. The dwell model shows the same pattern: validation MAE is 1.66 h against 3.07 h for the mean (−46%) and 1.98 h for Ridge, rising to 2.32 h after the shift.

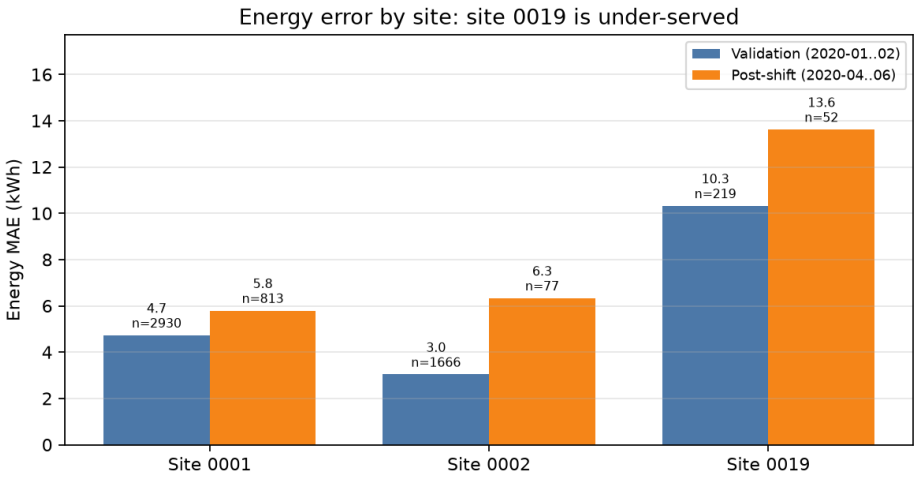


Figure 3. Energy MAE by site. Site 0019 has the fewest sessions and the largest error.

Per-site bias. Validation MAE is 4.73 kWh at site 0001 and 3.04 at site 0002, but 10.32 at site 0019 (R^2 −0.48 on 219 sessions). Site 0019 contributes under 2% of training rows, and only 45 of its reference-period sessions carry a positive energy request. The aggregate metric hides this; the per-site slice exposes it.

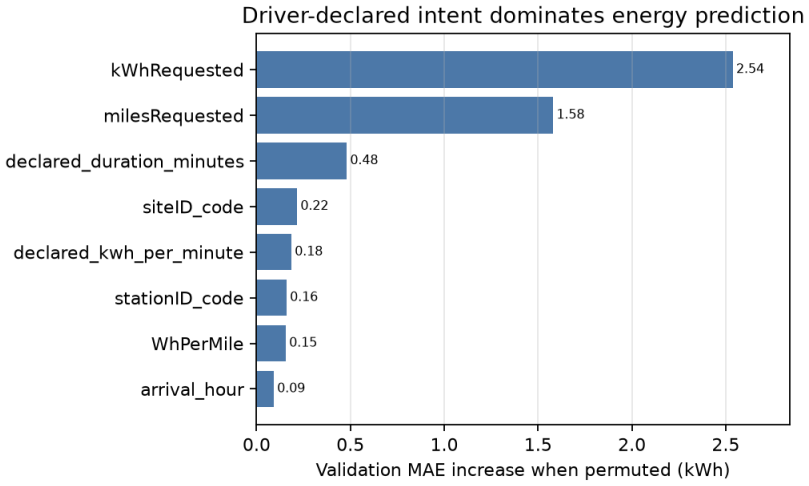


Figure 4. Permutation importance on validation: driver-declared intent dominates.

4.3 Registry, promotion gate and CI/CD

Each run registers its best-validation model and assigns the `staging` alias. A model reaches `production` only if its validation MAE beats both the mean and Ridge floors and is no more than 10% worse than the current Production model, scored on the same rows. Every decision is appended to `artifacts/promotion_decisions.jsonl`. GitHub Actions runs Ruff, 108 tests, the sample contract and pipeline, and a promotion-gate proof, then builds the Docker image. The proof registers a deliberately degraded candidate and fails the build if that candidate is promoted or the Production alias moves.

CI gate evidence	Value
Degraded candidate validation MAE	13.515 kWh (sample fixture; Production 6.342)
Failed checks	beats_mean_baseline, beats_ridge_baseline, production_regression_within_limit
Decision	**blocked** — Production energy and dwell aliases stayed at version 1

4.4 Serving, packaging and parity

FastAPI serves `/predict`, `/health` and `/metrics` from a Docker image with pinned dependencies. In Compose, the API loads `evcharge-sample-energy@production` from a local MLflow server, and an empty registry is bootstrapped by a one-shot container. Requests are validated by a Pydantic schema that rejects post-hoc fields. Each prediction is exported as Prometheus counters and histograms and appended to a JSON log of encoded features, without raw site or station identifiers. The API uses the same feature module as training, and a test rebuilds 1,000 sessions through the request path and asserts the feature matrices are identical. A live container smoke test checks the running stack.

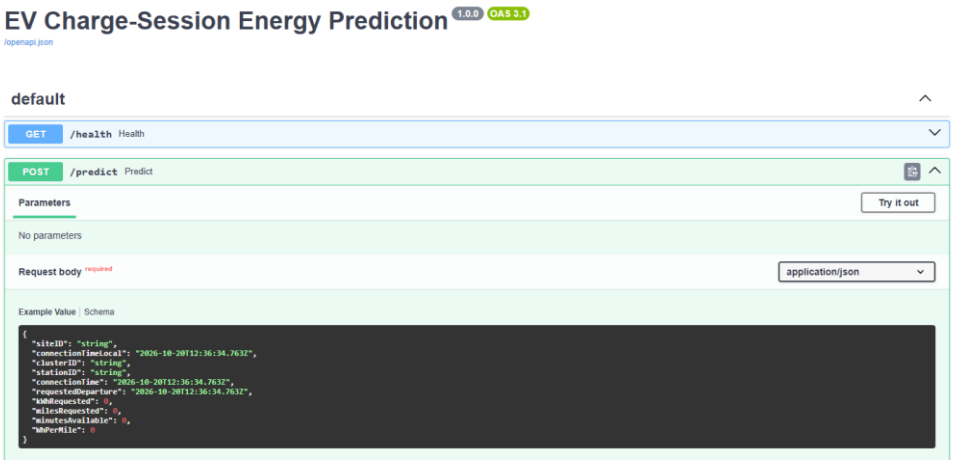


Figure 5. Served API contract (FastAPI /docs on the local Compose stack).

4.5 Drift monitoring and detector validation

A monthly Prefect flow fetches, normalises and validates each new or changed month. Content fingerprints let unchanged months skip work. The flow then scores the month against a checksum-verified frozen reference (2019-07 to 2020-02, 19,144 rows). It computes per-feature PSI and Jensen–Shannon distance, KS for numeric features, and target and prediction drift, all with sample counts and missingness. Scores go to Pushgateway, which Prometheus scrapes; alert rules fire on delivered-energy PSI > 0.2 and Grafana shows the dashboard. When the latest scored month breaches the threshold, the flow calls the quality-gated retraining flow directly; the Prometheus alert is the operator-facing signal.

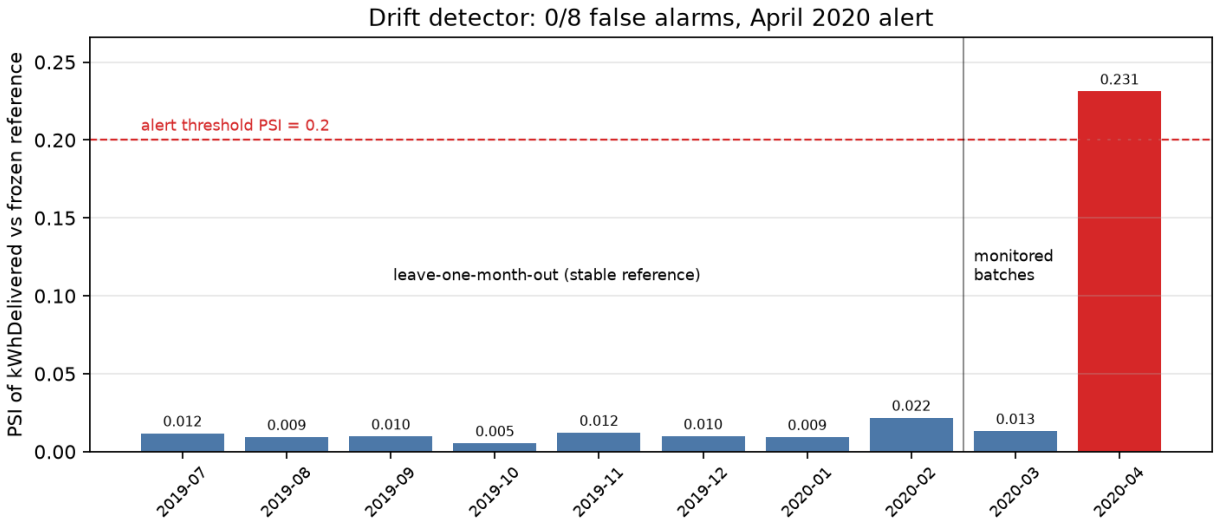


Figure 6. Delivered-energy PSI: leave-one-month-out stable months and the monitored 2020 batches.

Test	Result	Verdict
H1: April 2020 delivered-energy PSI	0.231 (KS 0.180, p = 3.5e−7; JS 0.164; 240 rows)	**Alert**, one month after March onset
H1: March 2020	PSI 0.013; volume −42% / −54%	No alert; not an early warning
False alarms (leave-one-month-out)	0 of 8 months; PSI 0.005–0.022	Pass
Sensitivity (injected energy shift)	+20% → 0.112; +30% → 0.209; +100% → 0.789	First alert at +30%
H2: request p90 (2021-05..09 vs reference)	Site 0002 +57.5%; 0001 −0.4%; 0019 +185.6% (45 ref. rows)	Mixed support
H3: declared-vs-delivered gap	Median 6.35 → 10.12 kWh; PSI 0.261	Supported

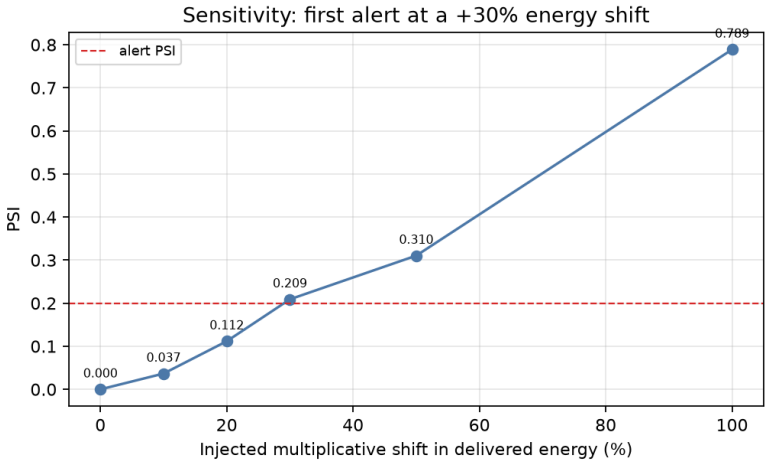


Figure 7. Controlled sensitivity: PSI against an injected multiplicative energy shift.

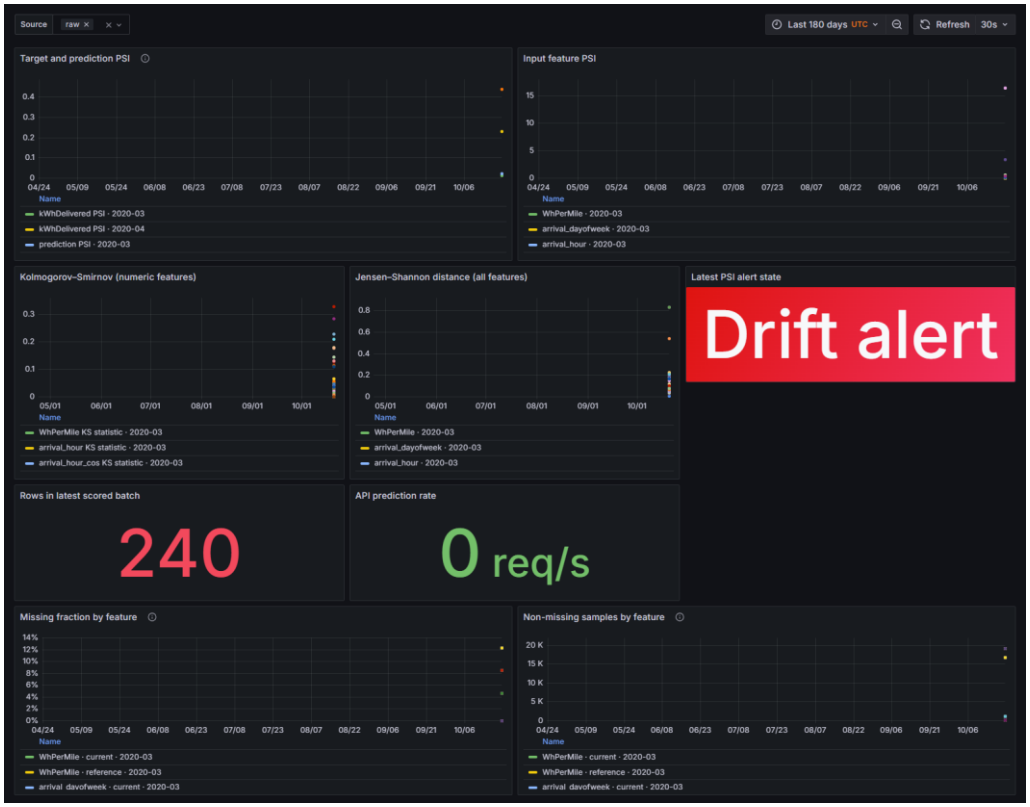


Figure 8. Grafana drift dashboard showing the April 2020 alert state (aggregate scores).

4.6 Retraining recovery: a measured null result

The recovery target was a post-shift MAE within 10% of the pre-shift 4.40 kWh, i.e. ≤ 4.84 kWh. I tested two chronological retraining windows, each with a held-out later test period excluded from fitting and selection. **Neither met the target.**

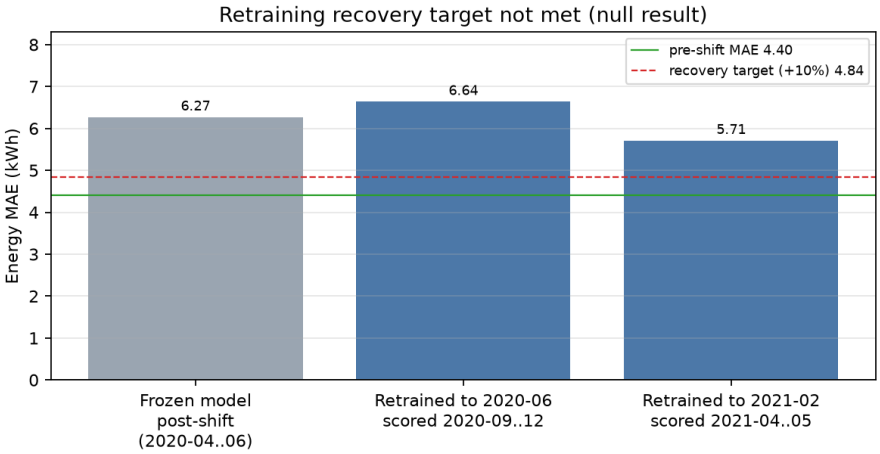


Figure 9. Post-shift MAE before and after chronological retraining against the declared target.

Trained through	Validation	Held-out test	Test rows	MAE	Within 4.84?
2020-06	2020-07..08	2020-09..12	869	6.641	No
2021-02	2021-03	2021-04..05	2,055	5.712	No (+29.8%)

Retraining narrows the gap as post-shift data accumulates, but a model trained mostly on the old regime does not recover pre-shift accuracy. The pipeline therefore does what it should: it detects the shift, blocks regressions, and allows rollback. It does not claim recovery it has not demonstrated.

4.7 Governance and rollback drill

The rollback runbook repoints the MLflow `production` alias to the known-good version and redeploys the previous image tag through the `EVCHARGE_IMAGE` variable. It ran on an isolated Compose project. Promoting v2 changed `/health` and `/predict` to v2. Rolling back restored v1 (prediction 10.49 kWh) and the captured previous image digest, and the user-facing stack was untouched. Because the drill reused the same model artifact, it verifies the mechanics, not model quality. `STACK.md` records each tool against named alternatives, together with a direct licence inventory.

5. Strategic Deductions & Business Impact

5.1 From error metrics to operations

The translations below use measured per-session errors. To size them, I assume the reference-period volume across the three sites: 19,144 sessions in 244 days, about 78 sessions per day. Summed absolute error is an upper bound on the daily energy a scheduler would hold back if it hedged every session by its typical error; real site-level errors partly cancel.

Lever	Before	After	Daily effect at ~78 sessions/day
Energy forecast vs naive average	7.89 kWh/session	4.40 kWh/session	≈ 272 kWh/day less error to buffer (≈ 616 → 344)
Ready-by estimate vs naive average	3.07 h/session	1.66 h/session	≈ 1.4 h closer per driver; ≈ 110 driver-hours/day
Undetected regime change	+1.87 kWh/session for months	Alert on first post-shift monthly batch	≈ 146 kWh/day of extra error made visible within a month
Bad release	Manual review	Automated gate in CI	A candidate 2.1× worse than Production never reached it
Recovery from bad release	Retrain and hope	Alias + image rollback	Known-good version restored without retraining

Converting buffered energy into currency needs each site's tariff, demand-charge rate and charger power, which ACN-Data does not provide. I therefore leave the impact in kWh and hours rather than invent a monetary figure. A site manager can multiply the avoided peak headroom (kW) by their own demand-charge rate.

5.2 Strategic deductions

- **Monitoring is the higher-value deliverable.** A model's accuracy decays without warning; a validated detector with no false alarms in eight stable months, plus release gates, protects every future model.
- **Declared intent is the main signal.** Driver requests dominate importance, and H3 shows the declared-vs-delivered gap widening by 3.8 kWh. Product changes to request capture directly affect forecast quality and should be monitored as a dependency.
- **Retraining is not a recovery plan.** A regime change needs new features or site-aware models, not only fresh data. Budget for model work after a major alert, not just a scheduled retrain.
- **Small sites need separate treatment.** Site 0019's validation error is 2.2–3.4× the other sites', so scheduling decisions there should use conservative defaults until more data or a site-specific model exists.

6. Limitations & Future Work

6.1 Honest constraints

- **Recovery target not met** (5.71 vs ≤ 4.84 kWh). The post-shift windows are small (869 and 2,055 rows), and behaviour kept changing through 2021.
- **Detector scope.** One real shift and coarse injected shifts were tested. PSI first alerts at about +30%, and monthly batching delays alerts by up to a month; March 2020 was not flagged.
- **Three Californian workplace and campus sites.** Results may not transfer to public, residential or depot charging.
- **Synthetic fixture.** It reproduces marginals but under-fits dwell time (median 5.08 vs 6.89 h), so sample-path metrics are a demonstration, not evidence.
- **Performance not load-tested.** API latency and throughput were not benchmarked beyond the smoke test.

6.2 Ethics and bias

- `userID` is pseudonymous but, combined with station and time, could reveal individual commuting patterns. The records stay out of the repository, the prediction log stores encoded features only, and a disclosed history finding was remediated.
- Per-site error is unequal. Drivers at the smallest site get worse ready-by estimates, so per-site slices should stay a release criterion rather than a report footnote.
- Sessions without driver requests (27%) are predicted less well. Any product that rewards accurate requests should check it does not disadvantage drivers who skip them.

6.3 What production would require

- MLflow with a database backend and object storage; images in a registry, referenced by digest.
- Authenticated, TLS-terminated ingress; secrets in a vault; network policy instead of loopback binding.
- Alertmanager routing to a staffed on-call rota with a named charging-operations approver for promotions.
- Daily or streaming drift windows with minimum sample sizes, plus a latency SLO and load testing.
- Model work: recency-weighted or site-aware training, an audited rolling-occupancy feature, and prediction intervals for scheduler headroom.

6.4 Changes from the accepted proposal

Proposal	Delivered	Reason
LightGBM	HistGradientBoosting	Windows OpenMP crash with pyarrow
Prefect 2	Prefect 3	Current supported major version; same flow model
SDV-generated drift scenarios	Deterministic multiplicative shifts	Exact, reproducible injected magnitude
Rolling site occupancy feature	Deferred	Point-in-time join not yet parity/leakage audited
Data snapshot submitted to upGrad	Data-free repository + manifests + free-token reproduction	Repository-only submission; respects publisher registration

Appendix A. Reproduction

Credential-free demo on the synthetic fixture (no token needed):

- `pip install -r requirements.txt` and `pip install -e ".[dev,orchestration,monitoring]"` (the CI install; Python 3.11)
- `python -m evcharge.pipeline --source sample`
- `python -m pytest -q`
- `docker compose up --build -d`

Licensed results (requires a free ACN-Data token as `ACN_API_TOKEN` in `.env`):

- `python -m evcharge.pipeline --source raw --fetch`, then compare `data_sha256` and split rows with `benchmarks/licensed_baseline.json`
- `python -m evcharge.validation.reference --source raw`
- `python scripts/evaluate_drift.py` and `python scripts/evaluate_retraining_recovery.py`
- `python scripts/make_report_figures.py` (regenerates the figures in this report)

Repository: https://github.com/deva-prakash-j/ev-charge-mlops

Appendix B. References

- Z. J. Lee, T. Li and S. H. Low. “ACN-Data: Analysis and Applications of an Open EV Charging Dataset.” Proc. 10th ACM International Conference on Future Energy Systems (e-Energy '19), 2019. https://ev.caltech.edu/dataset
- Project documentation: README.md, STACK.md, DATA_LICENCE.md, docs/architecture.md, docs/drift_hypotheses.md, docs/drift_evaluation.md, docs/operations_governance.md.

Appendix C. Detailed V2 architecture

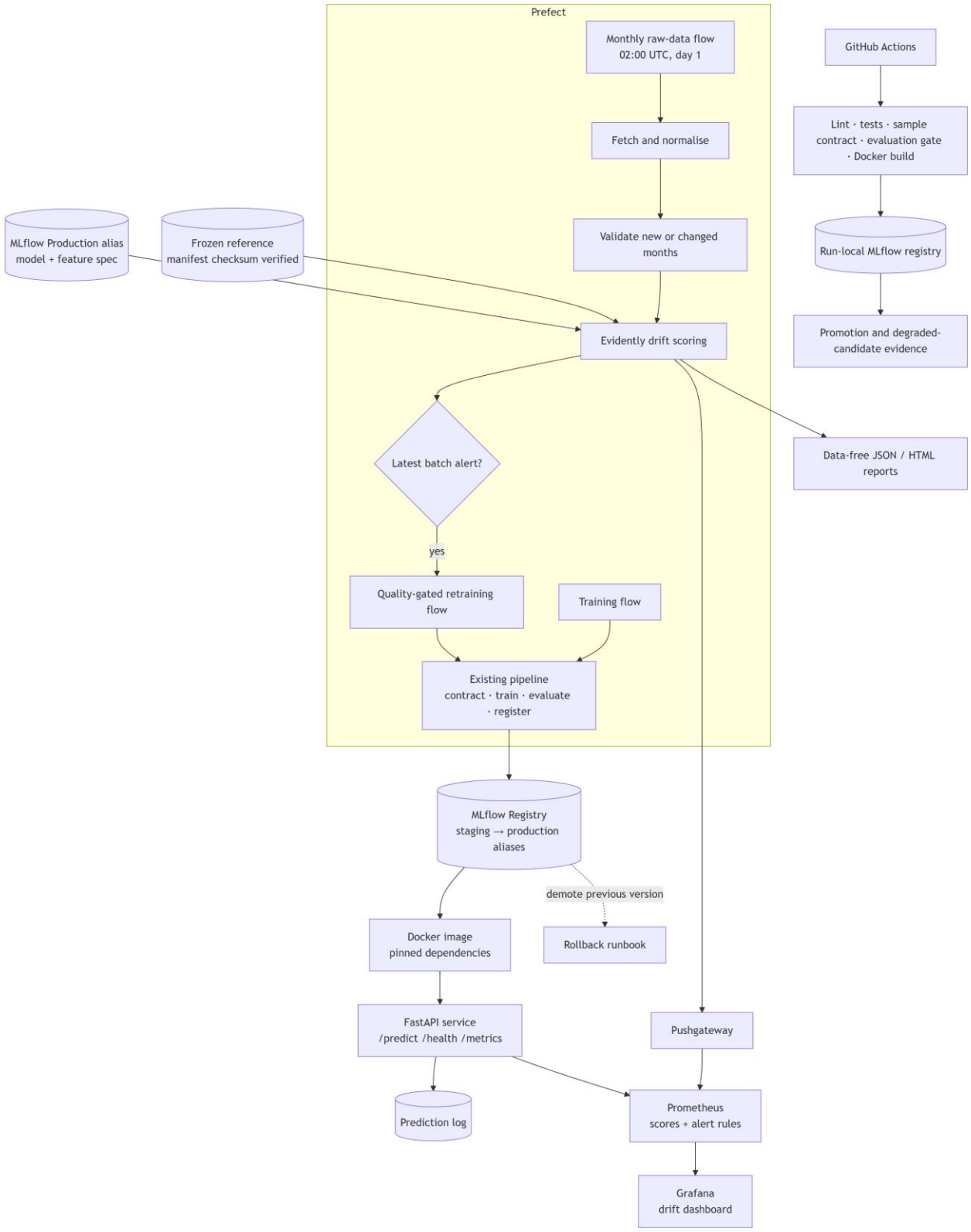


Figure C1. Implemented V2 architecture: ingestion, contract, training, registry, serving, monitoring and retraining (source: docs/architecture.md).